



Deployment and Infrastructure Architecture

TASCO Insurance × VETC Growth Platform — Architecture

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

1. Principles
2. Environments
3. Kubernetes topology
4. Network zones and Zero Trust
5. Ingress and WAF
6. Database: PostgreSQL high availability
7. Configuration
8. Secrets and configuration
9. CI/CD pipeline and promotion
10. Release strategies
11. Railway (UAT and pilots) and Docker Compose (local)
12. Capacity estimates
13. Cost-aware sizing
14. Disaster recovery
15. Observability

Document	Version	Date	Owner	Status
Deployment and Infrastructure Architecture	1.0	07 October 2026	iorta TechNXT Architecture	Issued for submission

The deployment assets are `Dockerfile`, `docker-compose.yml`, `deploy/k8s/`, `railway.json` and `.github/workflows/ci.yml`. This document is the **design** those assets implement. If a value here differs from a manifest, the manifest is authoritative for what is deployed, and this document states the intent. Raise a change to bring them back in line. Decision record: [ADR-012](#).

1. Principles

- One immutable artefact.** The same signed image (digest-pinned) is promoted from SIT to perf, prod and DR. Only configuration differs. The hosted UAT on Railway builds the same `Dockerfile` from the same commit.
- Twelve-factor.**
 - Configuration comes from the environment, and secrets from mounted files (`*_FILE`, `src/shared/config.js`).
 - Logs go to stdout as JSON.
 - Processes are stateless; state lives in Postgres.
 - API and jobs share one image with different commands (`node src/server.js`, `node src/jobs/cli.js <job>`).
- Least privilege everywhere.**
 - Containers run as the image's non-root `node` user (uid 1000) with a read-only root filesystem, all Linux capabilities dropped and the RuntimeDefault seccomp profile.
 - Network policy is default-deny.
 - The DB uses separate owner and app roles.
- Fail safe.**
 - Production refuses to boot without `JWT_SECRET`, `BLIND_INDEX_KEY`, `DATA_KEYS` or `DATABASE_URL` (no in-memory store in production), or with `DEMO_MODE` on (`loadConfig`).
 - Readiness stays false until the server listens and the DB answers.
 - `SIGTERM` handling drops readiness first, then drains, then exits within 25 s (`src/server.js`).
- Data residency.** Production personal data is stored and processed in Vietnam (**confirm with TASCO legal**: Cybersecurity Law 2018 and Decree 53/2022/ND-CP; Decree 13/2023/ND-CP and the Law on Personal Data Protection 2025 on cross-border transfer).

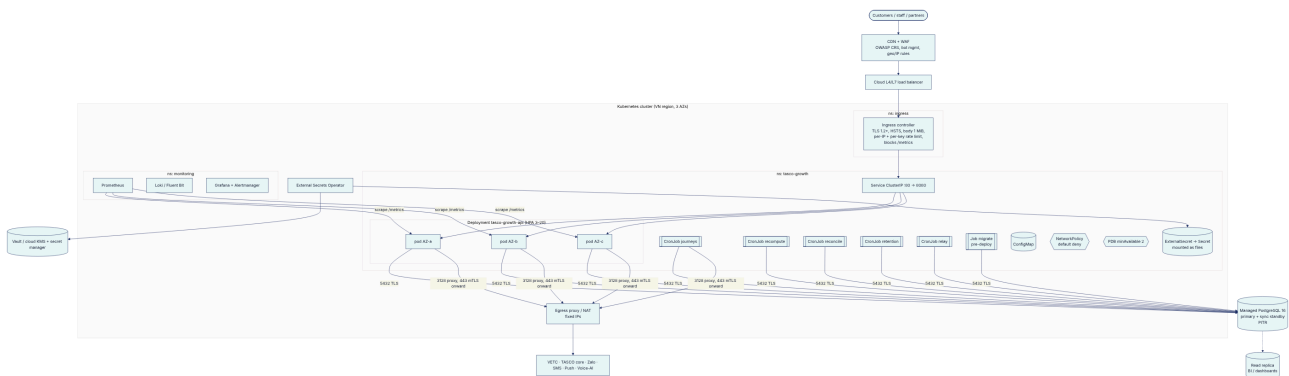
2. Environments

Env	Purpose	Hosting	Data	MODE: ENV / DEMO: MODE	Integrations	Promotion gate
dev	Developer laptop	<code>docker compose up</code> , or <code>npm run dev</code> (in-memory store)	Synthetic (SEED_RECORDS, SEED)	development / true (set by the scripts)	Sandbox adapters	—
CI	Pipeline tests	GitHub Actions + PostgreSQL service container	Synthetic	test / per test	Sandbox	All checks green
UAT (live)	Business acceptance, training	Railway, https://tasco-growth-api-uat.up.railway.app , with Railway PostgreSQL	Synthetic only	uat / true (UAT demo sign-in)	Sandbox adapters	Manual approval (product owner)
SIT	System integration with VETC/TASCO test systems	K8s namespace <code>tasco-growth-sit</code>	Synthetic + provider test accounts	production / false	Real adapters → provider sandboxes	Automatic on merge to <code>main</code>
perf	Load, soak and resilience tests at 6 M scale	Dedicated namespace sized like prod	Synthetic 6 M (bulk loader)	production / false	Sandbox gateways with latency/fault injection	Perf report signed off
prod	Live	K8s cluster in a Vietnam region, 3 AZs	Real personal data	production / false	Production providers	CAB approval + change window
DR	Warm standby	Second VN region / DC	Replica of prod	production / false	Production providers (failover config)	DR drill twice a year

Demo mode. `DEMO_MODE` is opt-in and defaults to `false`; production refuses it unless `ALLOW_DEMO_IN_PRODUCTION` is set. When on, it enables the demo sign-in helpers (`/api/demo/totp/:username`, the "Sign in with demo code (UAT)" button on the MFA screen, the `demoProfileId` customer login), simulated dates (`date`, `at`) and the `seed` job. The hosted UAT runs with `NODE_ENV=uat` and `DEMO_MODE=true`, plus the UAT-only settings `DEMO_PASSWORD`, `DEMO_TOTP_SEED` and `DEMO_ACCOUNT_SYNC`; its password is distributed in the access workbook and never written in documents. SIT, perf, prod and DR **must** run with `NODE_ENV=production` and `DEMO_MODE=false`; the Kubernetes ConfigMap sets both. Once provider UAT systems are available, UAT can move to a `tasco-growth-uat` Kubernetes namespace with real adapters.

3. Kubernetes topology

The production topology runs in one namespace per environment behind a CDN/WAF and an ingress controller.



3.1 Workload specification

Values below are those in `deploy/k8s/` (the manifests are authoritative).

Item	API Deployment	CronJob pods	Migration Job
Image	REGISTRY/tasco-growth-platform (pinned by digest in CD)	Same	Same
Command	node src/server.js	node src/jobs/cli.js <job>	node src/jobs/cli.js migrate (Argo CD PreSync hook)
Replicas	3; HPA min 3, max 20; CPU target 65 %, memory 75 %; scale-down stabilisation 300 s	1 per run	1
Requests / limits	250m CPU / 256 Mi → limit 1 CPU / 512 Mi	100m / 128 Mi → 1 CPU / 768 Mi	100m / 128 Mi → 1 CPU / 768 Mi
Node heap	NODE_OPTIONS=--max-old-space-size=384 in the image (75 % of the 512 Mi limit)	Same	Same
Probes	Liveness GET /health/live (period 10 s, failure 3); readiness GET /health/ready (period 5 s, includes a DB ping); startup probe up to 60 s	—	—
Security context	runAsNonRoot, runAsUser 1000 (the image's node user), readOnlyRootFilesystem, no privilege escalation, capabilities.drop: [ALL], RuntimeDefault seccomp, no token automount	Same	Same

Item	API Deployment	CronJob pods	Migration Job
Writable paths	emptyDir at /tmp only (64 Mi)	—	—
Spread	topologySpreadConstraints across zones	—	—
Termination	terminationGracePeriodSeconds: 30 (the app force-exits at 25 s)	backoffLimit: 1	backoffLimit: 2
Environment	PORT=8080, MIGRATE_ON_START=false, TRUST_PROXY=true, TRUST_PROXY_HOPS=1, DATABASE_SSL=true, DEMO_MODE=false	Same ConfigMap	Same ConfigMap

The image has no init process (no tini). The server handles `SIGTERM` and `SIGINT` itself (graceful drain). Where zombie reaping is needed outside Kubernetes, run the container with `docker run --init`.

3.2 CronJobs

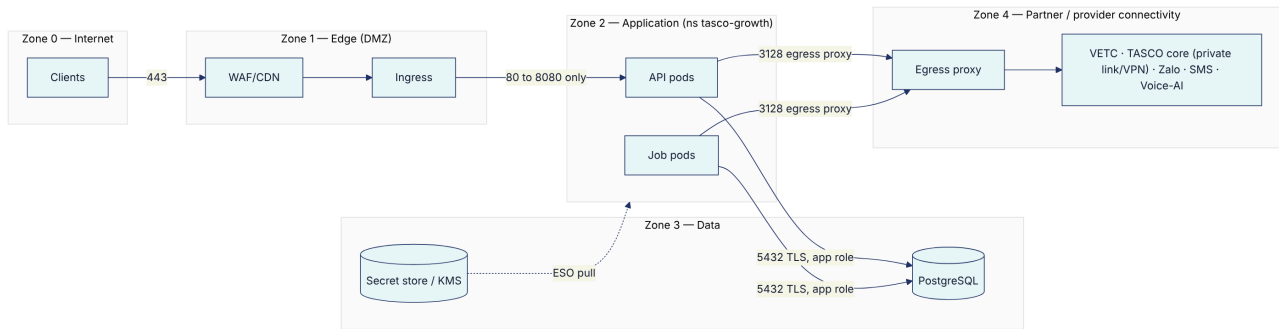
Job	Schedule (cron in UTC; ICT = UTC+7)	What it does (<code>job.yaml</code>)	Notes
journeys	15 1 * * * (08:15 ICT, inside the 08:00–20:00 contact window)	<code>journeyService.runDue</code> , then <code>events.drain()</code> . Pages through all due touchpoints; those blocked only by the contact window are deferred.	Add hourly runs within the window if volumes require it (<code>runDue</code> does not lock touchpoints, so runs must not overlap)
recompute	30 17 * * * (00:30 ICT)	Nightly full re-score so time-dependent facts (days) advance	At 6 M, shard (see section 12)
reconcile	0 19 * * * (02:00 ICT)	<code>opsService.reconcile</code> : completed orders ↔ payment ref ↔ policies; <code>pending_payment > 1 h</code> ; <code>payment_failed</code> ; <code>compensation_failed</code>	Alert if mismatches > 0
retention	0 20 * * * (03:00 ICT)	<code>opsService.applyRetention</code> (deletes <code>source_records > 365 d</code> and <code>voice_sessions > 180 d</code> ; other entities flagged for the archival pipeline)	—
relay	* / 5 * * * *	<code>events.drain()</code> , a safety net in addition to the 1 s in-process relay	—

All CronJobs use `concurrencyPolicy: Forbid`, `timeZone: Etc/UTC` and `backoffLimit: 1`. Short-lived job pods are not scraped by Prometheus. Job outcomes are recorded in `job_runs` (`GET /api/ops/jobs`) and in the JSON logs (`job finished`). Alert on the Kubernetes `kube_job_status_failed` metric.

3.3 PDB and disruption

`PodDisruptionBudget minAvailable: 2` for the API, so node drains and upgrades never take the service below 2 pods. Combined with readiness gating and graceful shutdown, rolling node upgrades cause no downtime.

4. Network zones and Zero Trust



Control	Implementation
Default deny	NetworkPolicy with empty ingress/egress in <code>tasco-growth</code> , then allow-lists: ingress from the <code>ingress-nginx</code> and <code>monitoring</code> namespaces to port 8080; egress to PostgreSQL (CIDR, port 5432), DNS (53) and the <code>egress-proxy</code> namespace (3128)
Identity per workload	Distinct ServiceAccounts for the API, each job and migrations; no token automount; DB credentials per workload type (the migrator owns the schema; the app has DML only, and <code>INSERT/SELECT</code> only on <code>audit_log</code>)
Never trust the network	TLS to Postgres with <code>rejectUnauthorized: true</code> (<code>postgresStore.js</code>); mTLS to providers; JWT verified on every request; the user reloaded per request (<code>identityService.authenticate</code>)
Admin access	No SSH to nodes. <code>kubectl</code> via SSO + MFA + just-in-time RBAC. DB break-glass via a bastion with session recording.
Service mesh (optional)	Istio/Linkerd mTLS between pods once more services exist. Not required for a single deployable.

5. Ingress and WAF

Control	Setting
TLS	TLS 1.2+ (prefer 1.3); modern cipher suites; certificate via cert-manager (ACME or corporate CA); HSTS is also set by the app in production
Forwarded headers	With <code>TRUST_PROXY=true</code> the app takes the client IP from the right-most trusted X-Forwarded-For hop (<code>TRUST_PROXY_HOPS</code> , default 1, <code>app.js#clientId</code>), so client-supplied values cannot spoof it. Set <code>TRUST_PROXY_HOPS</code> to the number of proxies in front of the app (for example 2 with a CDN in front of the ingress). The ingress should also overwrite XFF.
Rate limiting	Edge: the ingress sets <code>limit-rps: 50</code> per client IP (shared across replicas); add per-partner-key quotas and a stricter limit for <code>/api/auth/*</code> and <code>/api/customer/session</code> at the gateway. The app's in-process limiter (<code>RATE_LIMIT_MAX 300/min/IP</code> , <code>RATE_LIMIT_LOGIN_MAX 10/min/IP</code>) is defence in depth per replica , so the effective limit is × replicas.
Body size	1 MiB (matches <code>BODY_LIMIT_BYTES</code>)
Path rules	Block <code>/metrics</code> from outside the cluster (it can also require <code>METRICS_TOKEN</code>). Optionally restrict <code>/api/openapi.json</code> and staff APIs (<code>/api/*</code> except <code>/api/customer/*</code> , <code>/api/public/*</code> , <code>/api/partner/*</code> , <code>/api/auth/*</code> , <code>/api/meta</code>) to TASCO/VETC corporate egress IPs or ZTNA.
WAF	ModSecurity with the OWASP CRS enabled on the ingress (paranoia level 2 recommended on <code>/api/*</code>), with tuned exclusions for rules payloads (<code>/api/rules*</code> carries JSON Logic); bot management on customer and public endpoints
Partner mTLS (target)	A separate host <code>partner-api.<domain></code> with client certificate verification

6. Database: PostgreSQL high availability

Aspect	Design
Service	Managed PostgreSQL 16 in a VN region, or Patroni-managed on VMs/K8s operators (CloudNativePG) if no managed option meets residency
HA	Primary + synchronous standby in another AZ (automatic failover, about 30–60 s). Apps reconnect via the pg pool (failed queries surface as 5xx; idempotent retries are safe).
Backups / PITR	Continuous WAL archiving + daily base backups, encrypted with KMS. PITR retention 14 days in prod (35 days if the regulator requires it), plus monthly snapshots kept 12 months. Restore drill quarterly.
Read replica	Async replica for BI extracts, <code>/api/dashboard/*</code> (target: route via a second pool <code>DATABASE_READ_URL</code>) and analytics feeds
DR	Cross-region async replica (RPO ≤ 5 min), promoted in DR (section 14)
Connections	The pool per process is <code>DB_POOL_MAX</code> (default 10). With 20 API pods and 6 job pods, that is up to 260 connections. Put PgBouncer (transaction pooling) in front: advisory transaction locks and <code>SKIP LOCKED</code> are compatible with transaction pooling.
Parameters	<code>statement_timeout</code> 15 s set by the app; <code>idle_in_transaction_session_timeout</code> 60 s; <code>log_min_duration_statement</code> 500 ms; <code>pg_stat_statements</code> enabled
Roles	<code>tasco_owner</code> (migrations; owns tables, triggers); <code>tasco_app</code> (SELECT/INSERT/UPDATE/DELETE on collections; INSERT/SELECT only on <code>audit_log</code> ; no TRUNCATE; no DDL); <code>tasco_ro</code> (read replica, BI views)
Encryption	Storage encryption with KMS; TLS enforced (<code>hostssl</code>); PII is additionally field-encrypted by the app (ADR-006)
Partitioning (scale)	<code>messages</code> , <code>touchpoints</code> and <code>domain_events</code> by month (Solution Architecture, section 10) via a new numbered migration

7. Configuration

All settings come from `src/shared/config.js`. The ConfigMap holds non-secret values; secrets are mounted files.

Variable	Default	Prod value / note
<code>NODE_ENV</code>	development	production
<code>PORT</code>	3000	8080 (set in the image and the ConfigMap)
<code>PUBLIC_BASE_URL</code>	<code>http://localhost:3000</code>	<code>https://growth.<tasco-domain></code> , used in links and certificate URLs
<code>DATABASE_URL</code> / <code>_FILE</code>	— (memory store outside production)	Secret file ; required in production (<code>ALLOW_MEMORY_STORE</code> overrides, never in prod)
<code>DATABASE_SSL</code>	= production	true
<code>DB_POOL_MAX</code>	10	10 (with PgBouncer)
<code>JWT_SECRET</code> / <code>_FILE</code>	random in dev	Secret file , ≥ 48 random bytes
<code>JWT_TTL_SECONDS</code>	1,800	1,800 (900 for admin, once role-based TTL exists)
<code>MFA_REQUIRED_ROLES</code>	admin, rule_approver, compliance_officer, data_steward	Add <code>telesales_supervisor</code> , <code>partner_manager</code> and <code>support_engineer</code> (recommendation)
<code>BLIND_INDEX_KEY</code> / <code>_FILE</code>	random in dev	Secret file , 32 bytes base64

Variable	Default	Prod value / note
DATA_KEYS / _FILE	random dev key	Secret file , k1:<b64>, k2:<b64>
DATA_KEY_ACTIVE	last key	Explicit (e.g. k2)
CORS_ORIGINS	empty (same-origin only)	VETC/Zalo web origins if they call cross-origin
RATE_LIMIT_WINDOW_MS / _MAX / _LOGIN_MAX	60,000 / 300 / 10	Keep. The edge enforces the global limits.
LOCKOUT_MAX_FAILURES / LOCKOUT_MINUTES	5 / 15	Keep
BODY_LIMIT_BYTES	1 MiB	1 MiB
LOG_LEVEL	debug (dev) / info (prod)	info
DEMO_MODE	false (opt-in)	false
DEMO_PASSWORD, DEMO_TOTP_SEED, DEMO_ACCOUNT_SYNC	unset / unset / false	Unset (hosted UAT only)
ALLOW_DEMO_IN_PRODUCTION	false	false
SEED_RECORDS, SEED	2,500, 20261007	Unused when DEMO_MODE=false
SIM_TODAY	—	Unset (pins the business date; demos only)
RULES_DIR	config/rules	Default
TRUST_PROXY / TRUST_PROXY_HOPS	= production / 1	true / number of proxies in front of the app
METRICS_TOKEN / _FILE	unset	Secret file (bearer token for /metrics)
LINK_TTL_DAYS	30	30 (renewal link validity)
MIGRATE_ON_START	runs if not false	false in K8s (migration Job); Railway uses a pre-deploy command

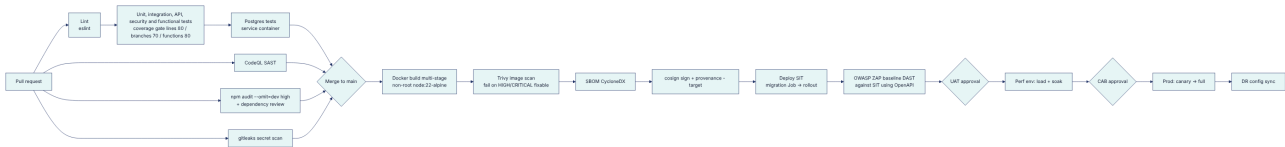
8. Secrets and configuration



Secret	Rotation	Procedure
DATA_KEYS	Yearly, or on suspicion	Add k(n+1) → set DATA_KEY_ACTIVE → rolling restart → run the re-key job (planned) → remove the old key after verification (ADR-006)
JWT_SECRET	Every 90 days	Rotation invalidates all sessions and all unexpired renewal links (the link key is derived from JWT_SECRET in <code>container.js</code>). Schedule off-hours, or implement multi-key verification (kid) first. Target: IdP-signed tokens (no shared secret).
BLIND_INDEX_KEY	Only on compromise	Requires recomputing *_bidx columns (planned job)
DB credentials	30–90 days	Dynamic DB credentials from Vault (database secrets engine) where possible
Provider credentials	Per provider policy	Store as files; adapters read on boot (restart to rotate)
Partner API keys	Partner-driven, ≤ 12 months	Issue the new key → partner switches → revoke the old one (DELETE /api/partners/keys/:keyId)

The rules: no secrets in images, Git, ConfigMaps or env literals. gitleaks runs in CI (§9). `*_FILE` is the only supported mechanism in K8s.

9. CI/CD pipeline and promotion



Stage	Gate (blocks promotion)
Lint, tests, coverage	Any failure; coverage below the gates (<code>npm run test:coverage</code>)
SAST (CodeQL)	New high/critical findings
Dependencies	<code>npm audit --omit=dev --audit-level=high</code> ; dependency review on new packages (any new runtime dependency needs an ADR, per ADR-002)
Secrets	Any gitleaks finding
Container	Trivy HIGH/CRITICAL with an available fix; image runs as root; missing SBOM
DAST	ZAP baseline high alerts
SIT smoke	<code>/health/ready</code> , login + MFA, quote → send → customer order with sandbox/provider test accounts
Prod	Signed image digest only (admission policy, e.g. Kyverno/Gatekeeper: <code>verifyImages</code>); CAB ticket reference

Promotion is **GitOps** (recommended: Argo CD or Flux). The pipeline commits the new image digest to the environment overlay (`deploy/k8s/overlays/<env>` if overlays are used), and the cluster reconciles. There are no `kubectl apply` calls from CI to prod.

10. Release strategies

Strategy	When	How
Rolling (default)	Backward-compatible code and config	<code>maxSurge: 1, maxUnavailable: 0</code> ; readiness-gated; PDB protects capacity
Canary	Risky behaviour changes (journeys, purchase)	Ingress weight split (NGINX canary annotations or Argo Rollouts): 5 % → 25 % → 100 %, with automated analysis on <code>http_requests_total{status=~"5.."} rate, p95 http_request_duration_seconds, integration_calls_total{result="error"}</code> and <code>orders_completed_total</code>
Blue-green	DB-affecting releases, major upgrades	Two Deployments (<code>-blue/-green</code>), with the Service selector switched after smoke tests. DB changes must be expand/contract : (1) migration adds columns/tables (compatible with old code) → (2) deploy new code → (3) a later migration removes old structures.

Caveats specific to this platform:

- **The outbox relay runs in every pod, including canary pods.** Old and new versions process the same event types, so handler changes must be backward compatible, or the event type must be versioned (`policy.issued.v2`).

- **CronJobs use their own image tag.** Update them in the same release, after the API rollout succeeds.
- **The rules cache TTL is 15 s, and rule sets are data, not deployments.** Rule changes go through maker-checker, not the release process.

11. Railway (UAT and pilots) and Docker Compose (local)

Railway (`railway.json`) hosts the live UAT at `https://tasco-growth-api-uat.up.railway.app`:

- Builder: Dockerfile. Start command `node src/server.js`. Pre-deploy command `node src/jobs/cli.js migrate`. Health check `/health/ready` (timeout 60 s). Restart policy on failure (maximum 5 retries).
`numReplicas: 2`.
- Railway PostgreSQL → `DATABASE_URL`.
- Secrets are Railway variables. `*_FILE` is not used there, so set `JWT_SECRET`, `BLIND_INDEX_KEY`, `DATA_KEYS` and `DATA_KEY_ACTIVE` directly.
- UAT settings: `NODE_ENV=uat`, `DEMO_MODE=true`, plus `DEMO_PASSWORD`, `DEMO_TOTP_SEED` and `DEMO_ACCOUNT_SYNC` (UAT only; see section 2).
- With 2 replicas, the in-process rate limiter and the logout denylist are per replica; credential and role changes still revoke tokens on both (persisted `tokensValidAfter`).
- Jobs can run as a Railway cron service with the same image and a `node src/jobs/cli.js <job>` command.
- **Constraints:** synthetic data only; no NetworkPolicy; hosting is outside Vietnam; not for production personal data (confirm with TASCO legal).

Docker Compose (`docker-compose.yml`): `app` (built from the Dockerfile, port 8080, waits for a healthy DB, read-only root filesystem, all capabilities dropped) + `postgres:16-alpine` with a named volume. It runs `NODE_ENV=development` with `DEMO_MODE=true` and local-only secrets. `docker compose up --build` gives a full stack with demo data; the local demo password defaults to `Tasco@Demo2026!`. Without Compose, `npm run dev` uses the in-memory store and needs no database.

12. Capacity estimates

Assumptions (to validate in the perf environment):

#	Assumption
A1	6.0 M vehicles; 7.8 M source records (1.3 per vehicle)
A2	Expiries uniform across the year: ≈ 16.4 k vehicles reach expiry per day
A3	About 30 % of vehicles are in an active journey; the renewal journey has 7 steps; on average 5 sends per expiring vehicle
A4	Voice bot is used for about 20 % of expiring vehicles (hot/warm + consent): ≈ 3.3 k calls/day
A5	Platform conversion ≈ 15 % of expiring vehicles: ≈ 2.5 k orders/day (1.2 lines/order)
A6	Staff users ≈ 300 (telesales heavy); peak customer app traffic follows push waves
A7	Average stored document sizes (jsonb + row overhead): profile 3 KB, lead 3 KB, source record 1.2 KB, touchpoint 0.6 KB, message 0.8 KB, voice session 4 KB, quote 3 KB, order 1 KB, policy 1 KB, audit 0.4 KB

Storage (year 1, primary, excluding WAL and backups)

Collection	Rows	Size
profiles	6 M	≈ 18 GB + 2.5 GB indexes
leads	6 M	≈ 18 GB + 2 GB
source_records	7.8 M	≈ 9.5 GB + 1.5 GB
touchpoints	≈ 7 M live + ≈ 42 M executed/yr	≈ 30 GB/yr (purge or partition executed rows > 90 d)
messages	≈ 30 M/yr	≈ 24 GB/yr
voice_sessions	≈ 1.2 M/yr (180 d retention)	≈ 2.5 GB steady
quotes / orders / policies	≈ 2.7 M / 0.9 M / 1.1 M per year	≈ 8 + 1 + 1.1 GB/yr
audit_log	≈ 18 M/yr (≈ 50 k entries/day)	≈ 7 GB/yr
domain_events	≈ 3 M/yr (purge done > 30 d)	< 1 GB steady
Total		≈ 120–130 GB in year 1 → provision 500 GB with autoscaling storage

Throughput

Workload	Estimate	Implication
API (customer + staff)	Baseline 10–20 RPS; push-wave peaks about 150 RPS for 15 min	3 pods are comfortable: the CI load smoke measured about 337 req/s from one process (25 concurrent, p95 about 103 ms, 0 % errors). HPA to 20 gives headroom for 1,000+ RPS.
Logins	300 staff around 08:00; scrypt ≈ 50 ms CPU each	Negligible; the login rate limit is 10/min/IP
Journey executions	≈ 115 k touchpoint evaluations/day (≈ 10 k per run if run hourly); ≈ 8 DB round-trips + 1 provider call each, sequential	≈ 15–25 min per run with providers at 100 ms. Fits the hour, with little margin. Required before full scale: parallel workers claiming touchpoints with <code>SKIP LOCKED</code> , and asynchronous voice calls (today <code>autoCall</code> is synchronous inside <code>runDue</code>).
Nightly recompute	6 M profiles × about 6 round-trips	Single-threaded about 6–10 h. Shard by <code>hash(id) % 8</code> with keyset paging to finish in about 1 h. Better: recompute only profiles whose journey window changes (days crossing a step offset).
Orders	≈ 2.5 k/day; peak ≈ 1 order/s	Bound by wallet and core latency, not by the platform
Outbox	≈ 10 k events/day	Trivial for the <code>SKIP LOCKED</code> relay
Audit appends	≈ 50 k/day (peak ≈ 20/s)	Well within serialised append capacity (≈ 1 k/s)

Database sizing (prod): primary 8 vCPU / 32 GB RAM / 500 GB SSD (≥ 6 k IOPS) + sync standby of the same size. Read replica 4 vCPU / 16 GB. Hot working set: profiles + leads + touchpoints indexes ≈ 10 GB, which fits in RAM.

13. Cost-aware sizing

Env	API	Jobs	Postgres	Notes
dev	local	local	memory or compose	\$0
SIT	2 pods (250m/512Mi)	on demand	2 vCPU / 8 GB, single AZ, 7 d PITR	Shared cluster; scale to zero at night (KEDA/cron scaler)
UAT	2 pods	on demand	2 vCPU / 8 GB	Shared cluster
perf	3–20 pods	sharded workers	8 vCPU / 32 GB, provisioned only for test windows	Ephemeral: create, test, destroy (IaC)
prod	3 pods baseline (HPA to 20)	≈ 5 pod-hours/day	8/32 primary + standby + 4/16 replica	Reserved/committed - use for the baseline; HPA for peaks
DR	0–1 pod warm (scale on failover)	—	Async replica 4/16	Warm standby, not active-active

Levers:

1. Async voice and journey workers instead of bigger pods.
2. Purge or partition high-churn tables (touchpoints, messages, events) to keep the DB on a smaller tier.
3. Run BI on the read replica, not the primary.
4. Use Vietnamese cloud providers if they meet InfoSec and residency requirements; they are typically cheaper than hyperscaler list prices.
5. Per-message channel costs (`config/rules/costs.json`) dominate infrastructure costs at scale. The NBA prefers push (cheapest) over ZNS over SMS over voice over telesales.

14. Disaster recovery

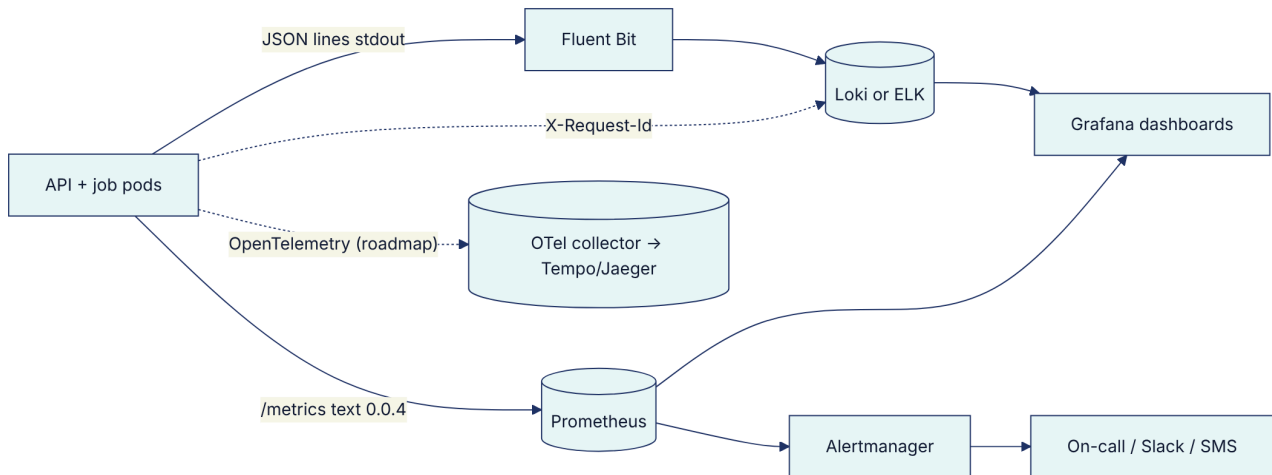
Target	Value
RPO	≤ 5 min (async cross-region replication + WAL archive)
RTO	≤ 1 h
Strategy	Warm standby: DR cluster manifests synced by GitOps with replicas = 0/1; DB replica in the DR region

Failover runbook:

1. Declare the incident (on-call + CAB emergency).
2. Promote the DR replica.
3. Point `DATABASE_URL` (secret) to the DR primary.
4. Scale up the DR API and CronJobs.
5. Switch DNS / global load balancer.
6. Verify `/health/ready`, `/api/audit/verify` (chain intact) and the outbox backlog.
7. Notify providers if the egress IPs change (pre-register DR IPs).

Drill twice a year. The PITR restore drill is quarterly.

15. Observability



Logs (`src/shared/logger.js`):

- One JSON object per line: `{ts, level, msg, service, ...fields}`.
- PII keys are redacted (`password, secret, token, authorization, cookie, phone, altPhones, name, totpSecret, apiKey, otp, code, mfaToken, refreshToken, accessToken`).
- Errors include the message, code and stack.
- Every API request logs `{requestId, method, route, status, ms}` (health, metrics and static requests are excluded).
- Retention: 30 days hot, 1 year archive. Security events are also in the audit chain.

Metrics (`src/shared/metrics.js` , exposed at `/metrics` , cluster-internal only, optionally protected by `METRICS_TOKEN`):

Metric	Labels	Use
<code>http_request_duration_seconds</code> (histogram)	route, method	Latency SLO (p95)
<code>http_requests_total</code>	route, method, status	Error-rate SLO, 401/403/429 anomaly detection
<code>integration_calls_total</code>	integration, result	Provider health
<code>integration_short_circuit_total</code>	integration	Breaker open → page
<code>events_published_total / events_processed_total</code>	type / type, status	Outbox flow; dead_letter → alert
<code>messages_total</code>	channel, status, journey	Send volume, blocked/failed rates (copy guard, consent)
<code>quotes_total, orders_completed_total</code>	channel; channel and journey	Business KPIs
<code>orders_compensation_total</code>	status	Saga compensations (<code>issuance_failed_refunded, compensation_failed</code>) → alert on <code>compensation_failed</code>
<code>voice_calls_total</code>	outcome	Bot outcomes, opt-out rate
<code>process_resident_memory_bytes, process_uptime_seconds</code>	—	Runtime health

Gaps:

- No `/metrics` from CronJob pods (use `job_runs` and logs, or a Pushgateway).
- No DB pool metrics.
- No event backlog gauge (available via `GET /api/ops/status`; add a gauge `outbox_pending` and `outbox_oldest_pending_seconds`).

Alerts (SLO-based):

Alert	Condition	Severity
API availability burn	5xx ratio > 1 % over 5 min (fast burn of the 99.9 % SLO)	Page
Latency	p95 > 1 s for 10 min on read routes	Ticket
Breaker open	<code>increase(integration_short_circuit_total[5m]) > 0</code> for wallet or core	Page
Dead letters	<code>increase(events_processed_total{status="dead_letter"}[15m]) > 0</code>	Page
Outbox lag	Oldest pending/processing event > 5 min	Page
Auth anomalies	<code>http_requests_total{route="POST /api/auth/login", status="401"} spike; status="423" (lockouts)</code>	Security
Reconciliation	<code>job.reconciliation mismatches > 0</code> (including <code>compensation_failed</code> orders)	Ticket (finance ops)
Audit chain	<code>/api/audit/verify ok=false</code> (synthetic probe hourly)	Security page
Job failure	<code>kube_job_status_failed > 0</code>	Ticket

Tracing: today correlation uses `X-Request-Id`, which is accepted from the client if it matches `^[\\w-]{8,64}$` and generated otherwise. It is echoed in the response and in error bodies and logged per request. Gap: it is not propagated into domain events (`correlationId` is null) or into outbound provider calls.

OpenTelemetry roadmap:

1. Instrument `node:http` and `pg` with OTEL auto-instrumentation (adds dev/runtime dependencies, which need an ADR-002 exception).
2. Propagate W3C `traceparent` to providers and put the trace id into `domain_events.correlationId`.
3. Generate span metrics (RED) in the collector, replacing the hand-rolled histograms.
4. Tail-based sampling that keeps all error traces.